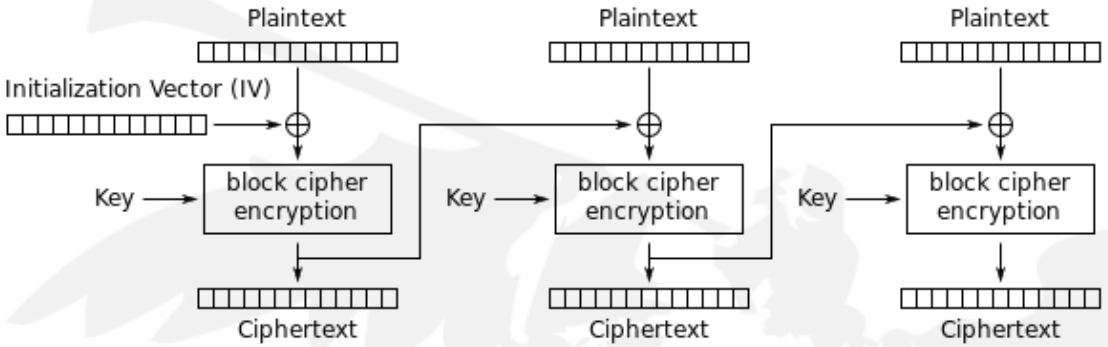
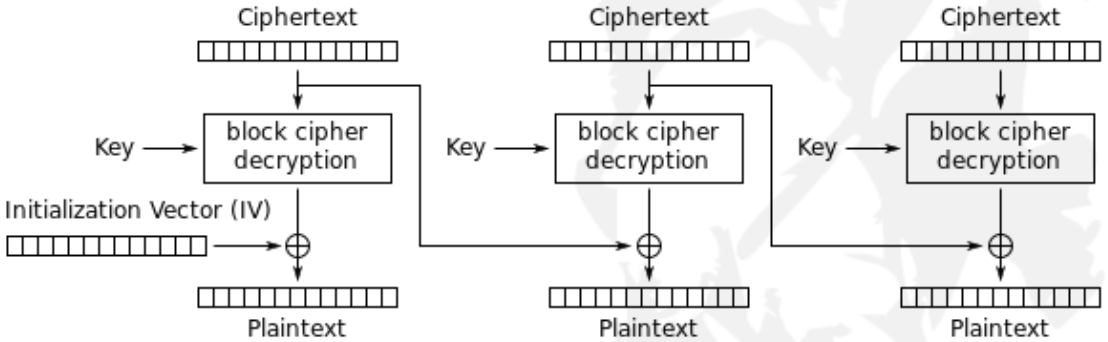
- **Padding Oracle Attack** *(ataque de oráculo de relleno)*
  - *Vulnerabilidad crítica que permite a un atacante descifrar información cifrada (AES-CBC) sin conocer la clave de cifrado*
- **Funcionamiento**
  - *El ataque se basa en el modo de funcionamiento CBC y en cómo se maneja el relleno (comúnmente PKCS#7)*
  - *Los atacantes envían consultas a un servidor, conocido como "oráculo", para determinar si un texto cifrado que envía al servidor se descifra en el servidor con relleno válido o inválido*
  - *El servidor actúa como un "oráculo" al dar una respuesta diferente (por ejemplo, un mensaje de error distinto, un tiempo de respuesta diferente o un código de estado HTTP distinto) en cada caso*
- **Recordemos el modo CBC y el relleno PKCS#7**

- **Modo CBC:** *Divide a nivel binario el mensaje en bloques de largo fijo, por ejemplo 128 bits. Necesita un* **Initial Value** *(IV) conocidos por emisor & receptor, que puede enviarse cifrado en modo ECB antes de enviar el mensaje*



Cipher Block Chaining (CBC) mode encryption

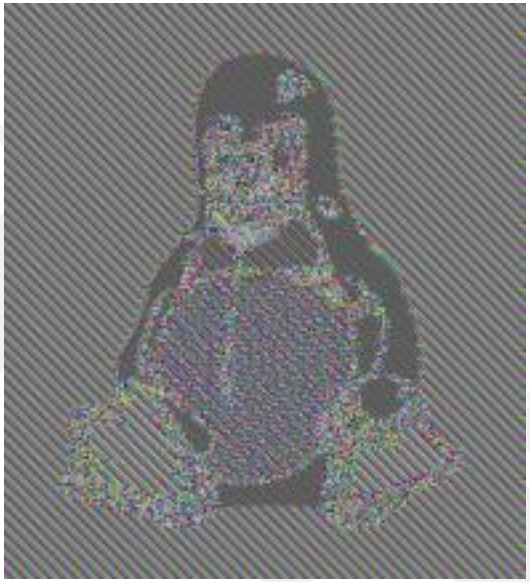


Cipher Block Chaining (CBC) mode decryption

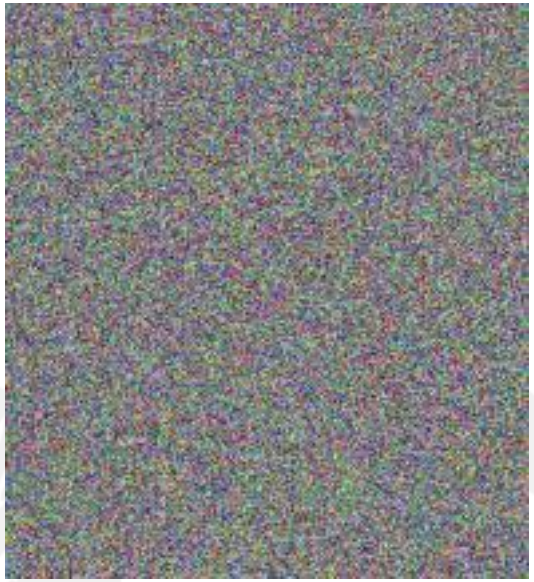
Implementación de la Confidencialidad.
Modos de operación



Original



Encrypted using ECB



Encrypted using CBC

Implementación de la Confidencialidad.
Modos de Cifrado

- ***Padding.*** *Cuando hay que cifrar menos bits que el tamaño del bloque (como los últimos bytes del mensaje), se añaden bytes adicionales para alcanzar tamaño del bloque* **(relleno)**
- ***PKCS#7****: Tipo de padding (Public-Key Cryptography Standards #7)*
  - *Para añadir el relleno*
    - **Determinar cuántos bytes faltan:** *Se calcula cuántos bytes se necesitan para alcanzar el tamaño del bloque (16 en el caso de AES)*
    - **Rellenar con el valor del conteo:** *Se añaden 'n' bytes, y cada uno de esos bytes tiene el valor numérico 'n'*
    - *Si no falta ningún byte se añade un bloque entero, con el valor del tamaño del bloque*
    - *Ejemplo: Faltan 4 bytes -> -- -- -- -- -- -- -- -- 04 04 04 04*

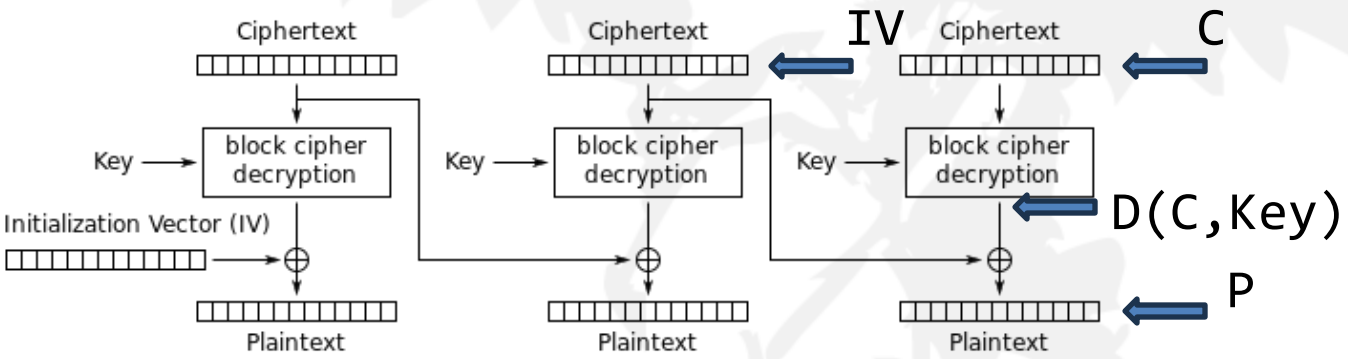
Implementación de la Confidencialidad.
Modos de Cifrado

- ***Padding.*** *Cuando hay que cifrar menos bits que el tamaño del bloque (como los últimos bytes del mensaje), se añaden bytes adicionales para alcanzar tamaño del bloque* **(relleno)**
- ***PKCS#7****: Tipo de padding (Public-Key Cryptography Standards #7)*
  - *Para eliminar el relleno*
    - **Se revisa el ultimo byte del ultimo bloque:** *Indica el número 'n' de bytes que son de relleno*
    - **Eliminar:** *Se eliminan 'n' bytes, y cada uno de esos bytes debe tener el valor numérico 'n', sino es así existe un error en el padding*

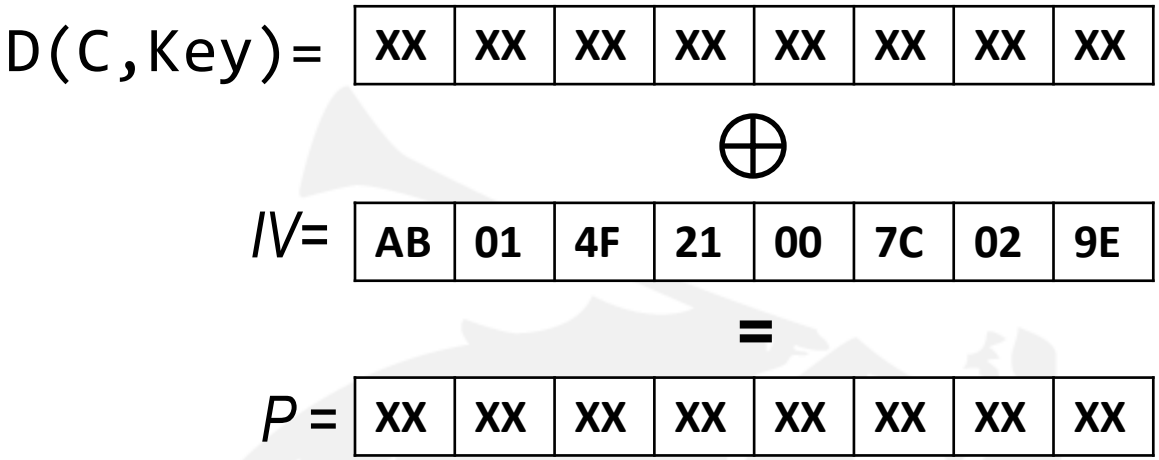
- *En el descifrado CBC, el texto plano (P) de un bloque se obtiene descifrando el texto cifrado (C) con la clave secreta y luego aplicando XOR con el bloque de texto cifrado anterior (IV):*

$$P = D(C, Key)\ XOR\ IV$$

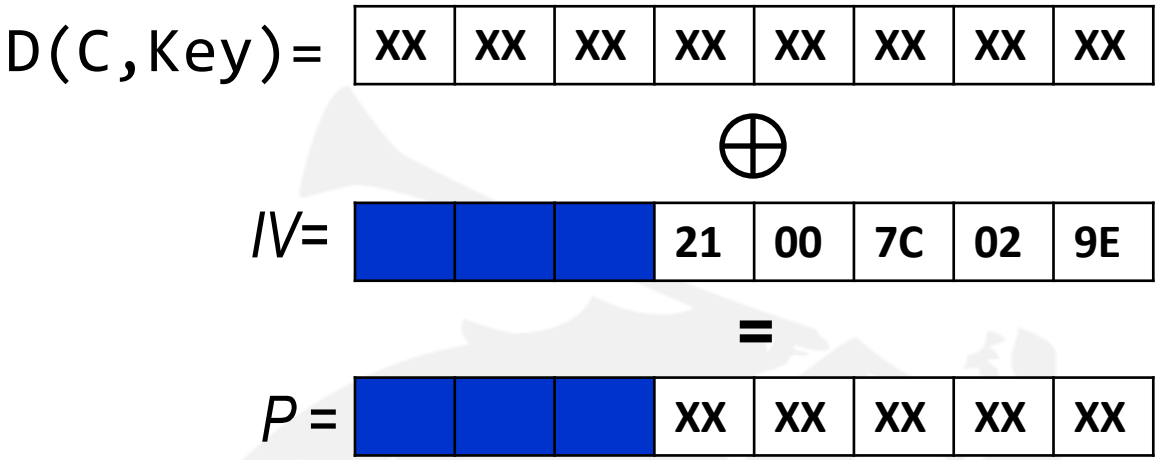
- *Primer ataque: Descubrir el tamaño del relleno (padding)*



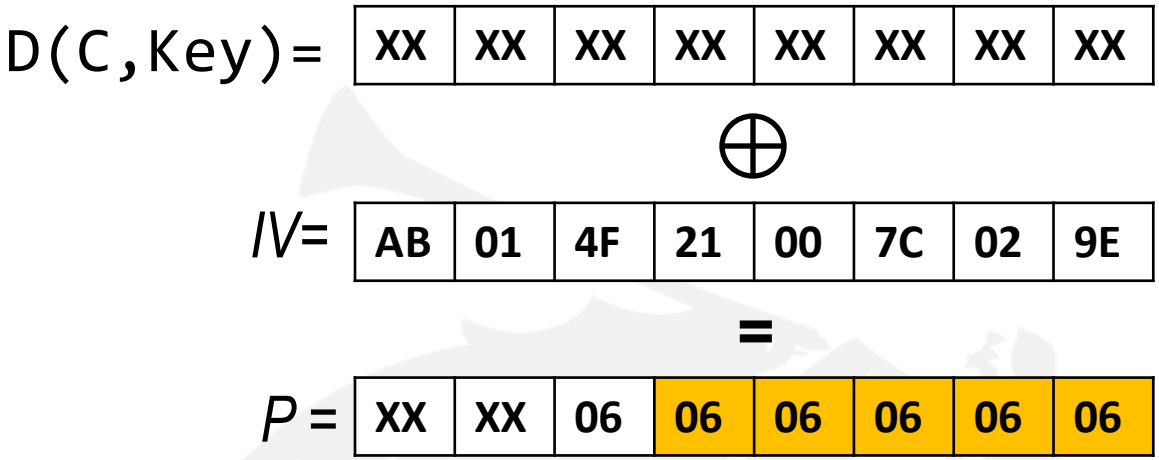
Cipher Block Chaining (CBC) mode decryption



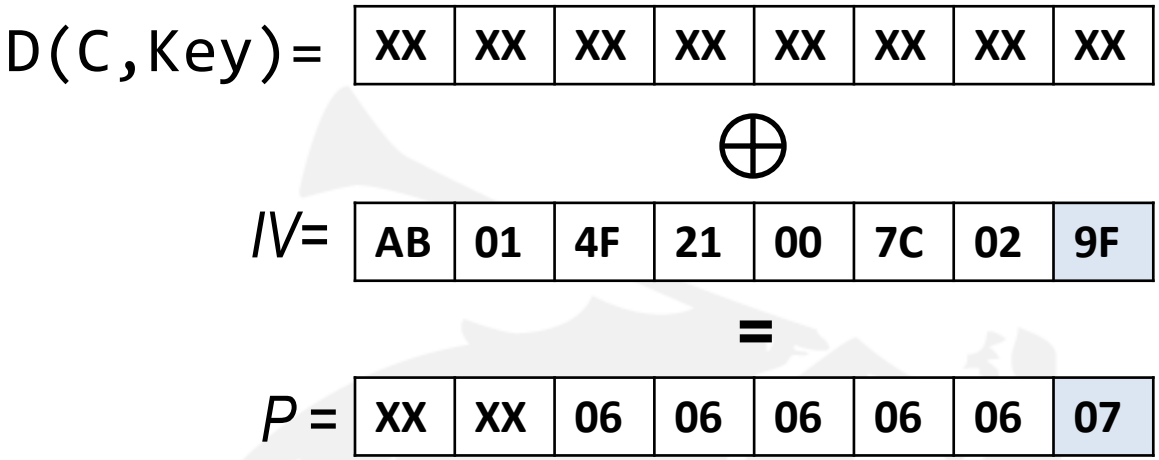
- *Vamos a modificar el primer Byte de IV con la intención de comprobar si el oráculo devuelve un error en el formato de padding*
- *Si se produce un error de padding será debido a que no todos los bytes que forman el padding del texto sin cifrar son iguales*
- *De esta forma podremos saber el tamaño de 'n' bytes del relleno*



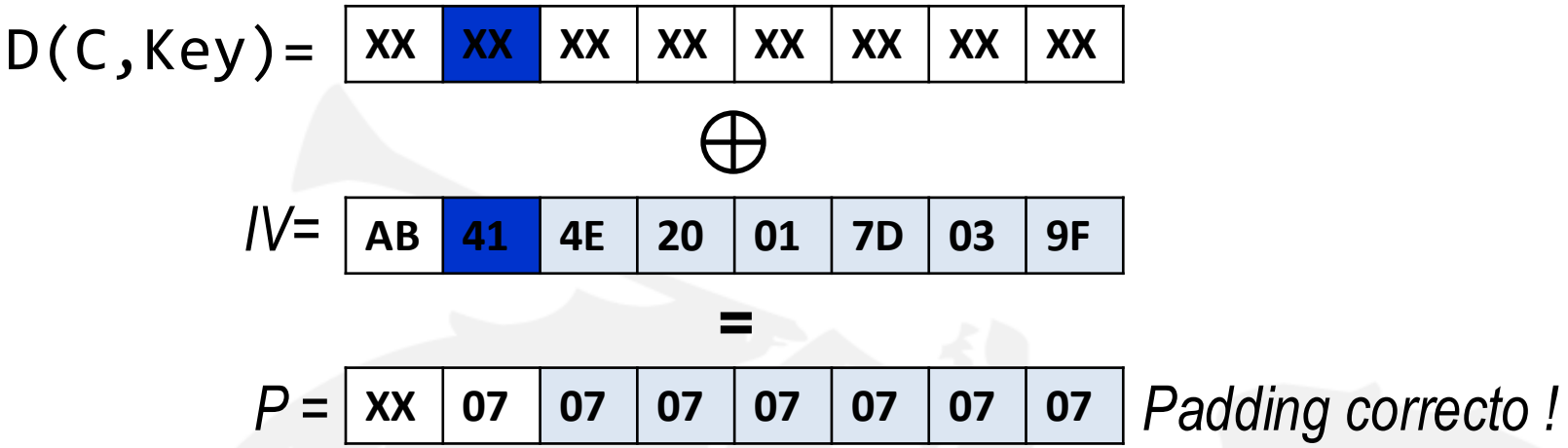
- *Ejemplo:*
  - *Supongamos que modificamos el primer byte de IV y no se produce ningún error de padding, y que lo mismo ocurre modificando el segundo byte de IV*
  - *Pero al modificar el tercer byte de IV el oráculo nos devuelve un error de padding*
  - *Podemos deducir que el tamaño del relleno es 6*



- *Para poder seguir obteniendo información con el oráculo, vamos intentar hacer creer al servidor que el mensaje tiene un relleno de 7 en lugar de 6*
- *Debemos modificar IV para que se genere en P un relleno de tamaño 7*
- *Ejemplo:*
  - *Para el ultimo byte del mensaje sabemos que 9E⊕06 = 98 = XX*
  - *Para cambiar el relleno a 07, debe cumplirse ?⊕07 = 98, por tanto el ultimo byte del IV debe cambiar a ? = 98⊕07 = 9F*



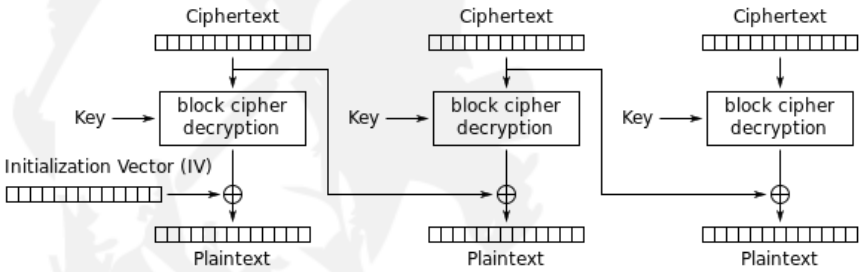
- *Ejemplo:*
  - *Repetiríamos el proceso para todos los bytes de IV asociados a los bytes de P que conocemos*



- *Ejemplo:*
  - *Para el byte marcado en azul, sabemos que algún valor combinado con XX debería dar 07, pero como no sabemos cuál es, probaremos desde 00 a FF hasta que el "oráculo" nos confirme que no hay error de relleno*
  - *Sabemos que XX⊕41 = 07, de lo que se deduce que XX = 41⊕07 = 46*
  - *Originamente teniamos un 01 en lugar del 41 en IV, por tanto 46⊕01 = 47 es el dato nos faltaba en P*

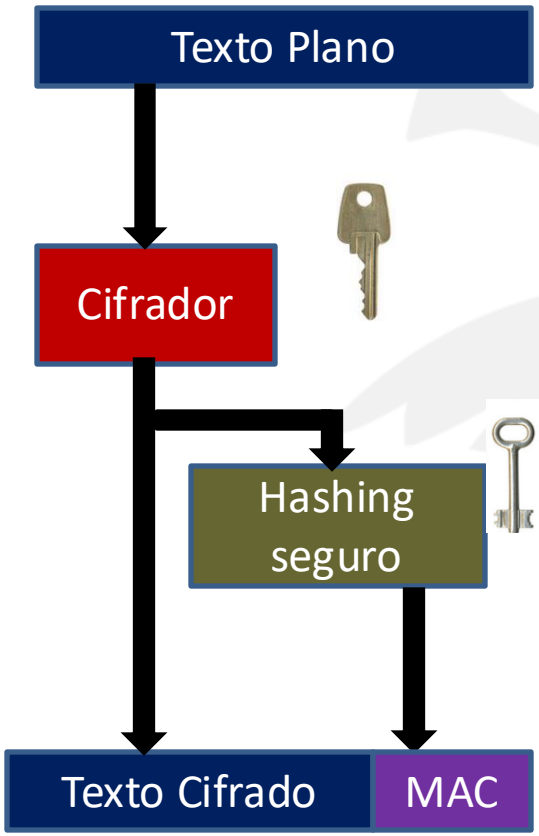
D(C,Key)=	XX	XX	XX	XX	XX	XX	XX	XX
	⊕							
IV=	AB	01	4F	21	00	7C	02	9E
	=							
P =	XX	47	06	06	06	06	06	06

- *El proceso se repetiría para el primer byte, ahora con padding 08*
- *Una vez acabado un bloque, se aplicaría el mismo proceso a los bloques anteriores, empezando por relleno del último byte (01)*
- *Sólo el primer bloque no se podría recuperar, ya que para ello sería necesario conocer el IV inicial*

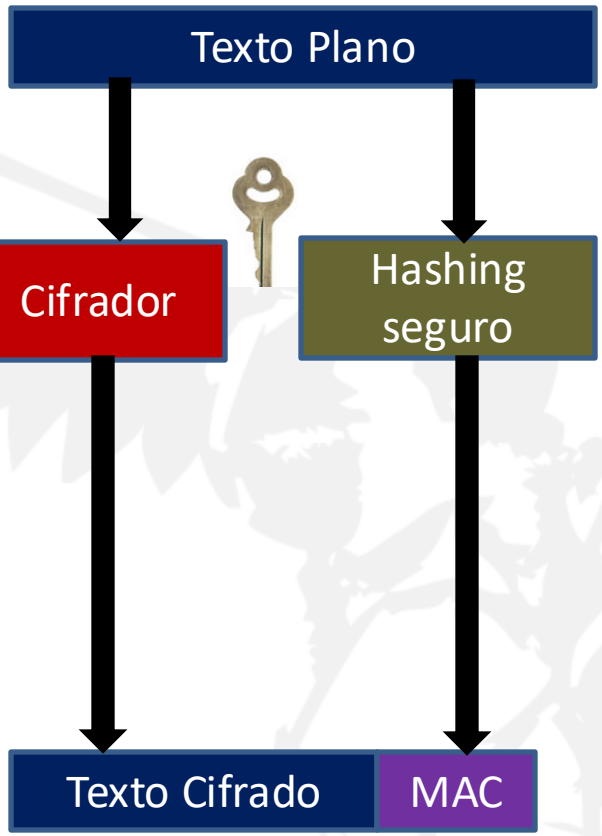


Cipher Block Chaining (CBC) mode decryption

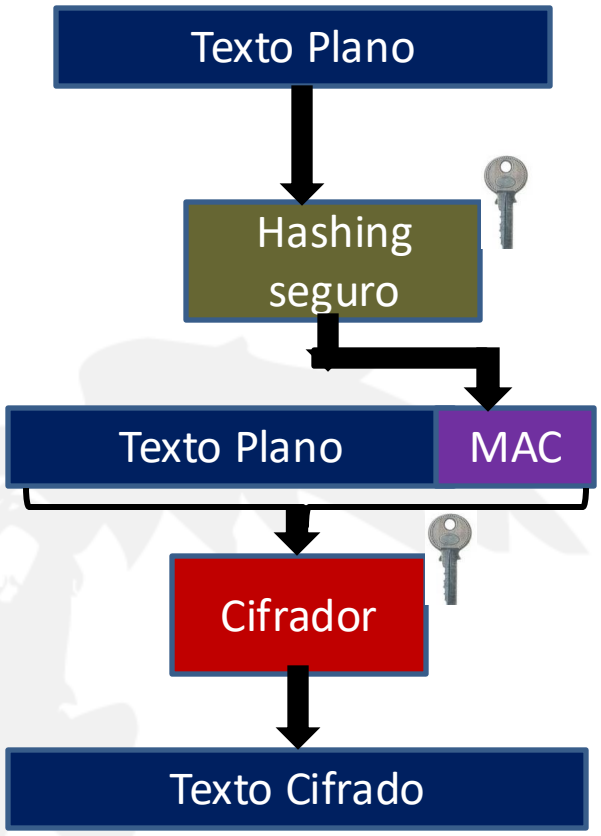
Implementación de la Confidencialidad.
Modos de cifrado



EtM



E&M



MtE

- **Modos de cifrado autenticado (AEAD)**. ***Authenticated encryption with associated data:*** *Cifrado Simétrico+MAC*
  - *Problemas: El modo de cifrado y el MAC usan la misma clave, puede disparar debilidades inesperadas*
  - *Solución: derive both the encryption key and the MAC key from a "master key" through a Key Derivation Function*
  - *Es mejor calcular el MAC en el texto cifrado, no del texto plano (previene ataques en modos de cifrados de bloques tales como CBC, puesto que se puede rechazar el descifrado de aquellos textos cifrados que el atacante haya alterados)*

- *Algunos modos relacionados son:*
  - **GCM (Galois Counter Mode):** *Libre de patentes, muy rápido y además soportado por TLS.*
  - **OCB (Offset Codebook Mode):** *El más rápido de todos. Con patentes para uso comercial.*
  - **EAX:** *Libre, fácil de implementar y ligero. Pero es lento en comparación con el resto.*
  - **CCM (Counter mode with CBC MAC):** *Implementado en el estándar WPA2.(Libre de patentes. Respecto a OCB es mucho más lento).*